

A 13-SWAP Layout for the Q-Synth VQE(8/71) Benchmark on Sycamore

Unsolved Labs
AI-generated research release

17 August 2026

Abstract

We give an explicit layout for the public Q-Synth VQE benchmark `vqe_8_4_10_100` on a frozen 54-qubit Sycamore coupling graph using 13 inserted SWAP gates. Shaik and van de Pol report, for the corresponding `vqe(8/71)` Sycamore row in SAT 2024, a Q-Synth2 timeout and a TB-OLSQ2 result using 16 SWAPs at reported depth 111. Our construction therefore improves that published comparison point from 16 to 13 inserted SWAPs, a reduction of 18.75%. The release does not prove that 13 is globally optimal or that a 12-SWAP layout is impossible.

The routed OpenQASM artifact is verified by a small exact symbolic checker. The checker tracks logical states through each physical SWAP, requires every single-qubit gate to retain its source parameter expression verbatim, and requires every routed CX to be the next source operation on both logical endpoint queues with the same control/target orientation. All 65 single-qubit gates and 71 CX gates are consumed exactly once, every two-qubit operation is checked against the frozen coupling graph, and the stored route schedule is replayed. No floating-point computation is part of this correctness oracle. An independent GF(2) checker verifies the CX+SWAP linear skeleton, while a full-unitary NumPy calculation on all 256 logical basis inputs is retained only as a secondary regression test.

1 Introduction

Quantum layout synthesis maps the logical qubits of a circuit to the physical qubits of a hardware coupling graph and inserts routing operations so that two-qubit gates act only on adjacent physical qubits. Because a SWAP can be decomposed into three CX gates, reducing inserted SWAPs directly reduces a common two-qubit-gate overhead. Exact layout synthesis is difficult on large processors, motivating SAT-, SMT-, planning-, and heuristic-based methods.

Shaik and van de Pol introduced Q-Synth2, a SAT formulation based on parallel plans that can optimize inserted SWAP count for deep circuits on processors with more than 100 physical qubits [1]. Their SAT 2024 Table 3 reports results for VQE circuits on Sycamore, Rigetti, and Eagle. For the 8-logical-qubit, 71-CX instance denoted `vqe(8/71)`, Q-Synth2 times out on Sycamore while TB-OLSQ2 returns a 16-SWAP solution at reported depth 111.

This release fixes the exact public Q-Synth source file and gives a 13-SWAP construction for a frozen 54-node, 88-edge undirected Sycamore graph. The load-bearing contribution is constructive: a concrete route is provided and checked. No lower-bound certificate is supplied, so the result must not be read as a proof of 13-SWAP optimality.

A second goal of this release is verification clarity. Earlier versions of the repository regenerated single-qubit gate angles through binary floating-point evaluation before serializing the mapped QASM. Although the resulting numerical error was tiny, that approximation was unnecessary. The

current artifact preserves each OpenQASM parameter expression symbolically and uses an exact dependency/placement checker for the complete circuit.

2 Frozen benchmark and routing model

The logical circuit is the Q-Synth file

`Benchmarks/SAT-24/VQE/vqe_8_4_10_100.qasm`

at repository commit

`95a820e16ac578289ea692ce8665afb48788892d`

Its Git blob SHA-1 is

`cdbc957d2c8f9a9f25fa5a530b80d8b6e7bd8af5`

The circuit has eight logical qubits, 65 single-qubit u gates, and 71 directed CX gates.

The physical device model is the 54-vertex, 88-edge undirected graph stored in `benchmark.json`. The routing semantics are:

1. an injective initial placement maps the eight logical qubits to physical vertices;
2. a CX may execute only when the current physical locations of its two logical operands are adjacent;
3. operations preserve ordinary program order on each logical qubit, while operations on disjoint logical qubits may interleave;
4. a SWAP may be inserted only on a hardware edge and is the only operation that changes the placement;
5. unused physical vertices may carry routing ancillas; and
6. the reported optimization quantity is the number of inserted SWAPs.

The release does not use bridge gates, teleportation, measurement, qubit reuse, calibration weights, or relaxed CX commutation rules.

3 The 13-SWAP construction

The initial logical-to-physical placement is

logical q	0	1	2	3	4	5	6	7
physical p	46	53	41	40	47	52	34	29.

The inserted SWAP sequence is

$(46, 53), (40, 46), (41, 47), (41, 47), (47, 53),$
 $(34, 41), (41, 47), (40, 46), (45, 52), (41, 46),$
 $(40, 46), (40, 46), (34, 41).$

Between those SWAPs, the 71 source CX operations are scheduled exactly as recorded in `route.json`. The final placement is

$(0, 1, 2, 3, 4, 5, 6, 7) \mapsto (46, 34, 41, 40, 53, 45, 47, 29).$

Construction claim. Under the frozen routing model, this is a valid route using exactly 13 inserted SWAP gates.

The route checker derives the ordered CX list from the pinned source QASM, checks it against the frozen benchmark metadata, reconstructs the per-logical-qubit dependency DAG, validates every SWAP edge, and reconstructs the stored CX schedule.

Under the conventional three-CX decomposition of a SWAP, 13 SWAPs correspond to 39 added CX gates, compared with 48 for a 16-SWAP solution. Thus the construction uses 3 fewer SWAPs, or 18.75% fewer inserted SWAPs, than the published 16-SWAP comparison point.

The emitted circuit has native instruction-DAG depth 110 according to the repository’s deterministic ASAP checker. SAT 2024 reports depth 111 for the TB-OLSQ2 16-SWAP row, but the exact baseline routed circuit is not frozen in this release. We therefore do not use the one-unit depth difference as a load-bearing comparison under byte-identical tooling.

4 Exact full-circuit equivalence

The exact checker works directly with the symbolic OpenQASM instructions.

For the source circuit, let each operation receive a unique identifier, and let Q_i be the queue of source operations touching logical qubit i , in source order. The mapped circuit is processed from left to right while maintaining a placement map M from logical qubits to physical vertices.

For an inserted SWAP on physical vertices a, b , the checker verifies that $\{a, b\}$ is a hardware edge and updates M by exchanging the logical states, if any, currently carried by a and b .

For a mapped single-qubit instruction

$$\mathbf{u}(\theta, \phi, \lambda) \mathbf{q}[p],$$

the checker determines the logical qubit i currently carried at p . The next entry of Q_i must be a source $\mathbf{u}$ operation on i with the same three parameter expressions after removing only insignificant whitespace. Expressions such as $\mathbf{pi}/2$ are not numerically evaluated.

For a mapped CX from physical control p_c to physical target p_t , both vertices must be adjacent in the frozen graph. If they currently carry logical qubits i and j , respectively, then the next entries of both Q_i and Q_j must be the same source operation and that operation must be the directed source CX $i \rightarrow j$.

Exact-equivalence theorem. If the checker consumes all source queues, then the mapped circuit implements the source circuit exactly, with logical outputs located according to the reported final placement.

Proof. Maintain the invariant that, after each mapped operation, the quantum state on every occupied physical wire equals the state of the logical qubit indicated by the tracked placement after exactly the source operations already consumed from its queue. A physical SWAP preserves the logical state while changing only its location. A checked single-qubit or CX instruction is exactly the next source operation on every logical wire it touches, so it preserves the invariant. Operations that are reordered relative to the source touch disjoint logical wires and therefore act on disjoint tensor factors and commute. Induction over the mapped instruction list proves the claim. At termination all source operations have been consumed, yielding the source unitary embedded under the final placement. $\square$

This proof does not depend on floating-point evaluation or on the stochastic search procedure that found the route.

5 Independent and secondary checks

GF(2) structural check. The dependency-free script `verify_linear_equivalence.py` interprets the source CX network and the mapped CX+SWAP network as exact linear maps over $\mathbb{F}_2$. It verifies equality of the logical linear transformation and checks that the routing ancilla carries no residual linear garbage. This is an independent structural check, but by itself it does not establish equivalence of arbitrary single-qubit gates.

Numerical unitary regression. The build script additionally evaluates the frozen `u` parameter expressions in double precision and compares all 256 logical computational-basis columns on the nine active physical nodes. This test is useful for regression detection but is explicitly secondary to the exact symbolic checker.

Depth check. `compute_qasm_depth.py` assigns every instruction to the earliest layer after all previous instructions on its participating physical qubits. Under that repository-local definition, the source depth is 102 and mapped depth is 110.

6 Reproducibility and trust boundary

The exact checks require only Python 3.12 and the standard library:

```
python verify.py
python verify_exact_qasm_equivalence.py
python verify_linear_equivalence.py
python compute_qasm_depth.py original_vqe_8_4_10_100.qasm mapped_route.qasm
```

The numerical regression additionally uses the pinned NumPy version in `requirements.txt`:

```
python -m pip install -r requirements.txt
python build_and_verify_full_qasm.py route.json
```

The build command regenerates `mapped_route.qasm` from the pinned source and route while preserving source angle expressions verbatim. CI requires that regeneration leave the committed artifact unchanged.

The correctness boundary trusts the small committed exact checkers, the standard semantics of the supported OpenQASM gates, the frozen source blob, and the frozen coupling graph. It does not trust the search procedure or AI generation as proof. The numerical simulation is not a correctness oracle.

7 Relation to the published comparison

The SAT 2024 paper’s Table 3 labels the Q-Synth2 column as SWAP-optimal when a solution is returned and TB-OLSQ2 as near-optimal [1]. For `vqe(8/71)` on Sycamore, Q-Synth2 times out and TB-OLSQ2 reports 16 SWAPs and depth 111. Consequently the public paper supports a comparison point, not a proof that 16 is optimal for this row.

R005 therefore makes a deliberately narrower claim: it provides a verified 13-SWAP construction and improves that published 16-SWAP point. It does not infer a global lower bound from the comparison source.

8 Limitations

The present release does not include:

- a proof that 12 or fewer SWAPs are impossible;
- a hardware experiment or a calibration-aware noise model;
- an independently reproduced copy of the exact published 16-SWAP routed circuit;
- a claim that the local ASAP depth checker is identical to the implementation used to populate the paper’s depth column; or
- completed independent external specialist review.

A future global optimality claim would require a valid lower-bound certificate, for example an independently replayable UNSAT/proof-producing exact-search certificate for all routes with at most 12 SWAPs under the same frozen model.

9 Release provenance

The source benchmark is pinned by repository commit, path, and Git blob SHA-1. The full provenance and statement crosswalk are recorded in `SOURCE_AUDIT.md` and `STATEMENT_AUDIT.md`, respectively.

This is an AI-generated Unsolved Labs research release. AI generation and search are treated as discovery mechanisms, not as correctness evidence.

References

- [1] I. Shaik and J. van de Pol. Optimal Layout Synthesis for Deep Quantum Circuits on NISQ Processors with 100+ Qubits. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*, LIPIcs 305, Article 26, 2024. DOI: 10.4230/LIPIcs.SAT.2024.26.
- [2] A. W. Cross et al. OpenQASM 3: A Broader and Deeper Quantum Assembly Language. *ACM Transactions on Quantum Computing*, 3(3), 2022.
- [3] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*, 10th Anniversary Edition. Cambridge University Press, 2010.